第8回：最新ASRモデル

Conformer / Whisper など大規模 End-to-End 認識

小野原彩香｜北星学園大学

📌 Zoom録画を後日公開します。課題はメール/Google Driveでも提出可。

本日の到達目標

1. End-to-End モデルの概念と従来型との違いを説明できる
2. CTC・Attention-based Encoder-Decoder の仕組みを理解する
3. Conformer と Whisper のアーキテクチャ的特徴を説明できる
4. OpenAI Whisper を使って実際に音声認識を行える

従来型 vs End-to-End モデル

項目	従来型（GMM-HMM）	End-to-End
構成	音響モデル＋言語モデル＋デコーダ	単一ネットワーク
入力	音声特徴量（MFCC等）	音声波形 or メルスペクトログラム
出力	音素列→単語列	直接テキスト
学習	各モジュールを別々に訓練	エンドツーエンドで一括
アライメント	必要（強制アライメント等）	不要（モデルが暗黙的に学習）

> 学習データは（音声, テキスト）のペアだけでOK！

代表的な E2E アーキテクチャ

```
[入力: 音声波形]
        ↓
[ニューラルネットワーク（1つのモデル）]
    ・音声の特徴を自動抽出
    ・音素への変換を暗黙的に学習
    ・言語的な文脈も考慮
        ↓
[出力: 文字列 / 単語列]
```

主な E2E アーキテクチャ：

1. **CTC**（Connectionist Temporal Classification）
2. **Attention-based Encoder-Decoder（AED）**
3. **RNN-T**（Transducer）
4. これらのハイブリッド

CTC の仕組み — アライメント問題の解決

問題：音声フレーム数 T（例: 300）と出力文字数 L（例: 20）が一致しない。

解決策：ブランクラベル ϵ を追加し、多対一マッピングで変換。

```
目標文字列 "cat" に対するCTC出力パスの例：
  c c a a t t  →  繰り返し除去  →  cat
  c _ a _ t _  →  ブランク除去  →  cat
  _ c a t _ _  →  ブランク除去  →  cat
```

変換ルール：(1) 繰り返しをまとめる　(2) ブランクを除去

CTC 損失関数

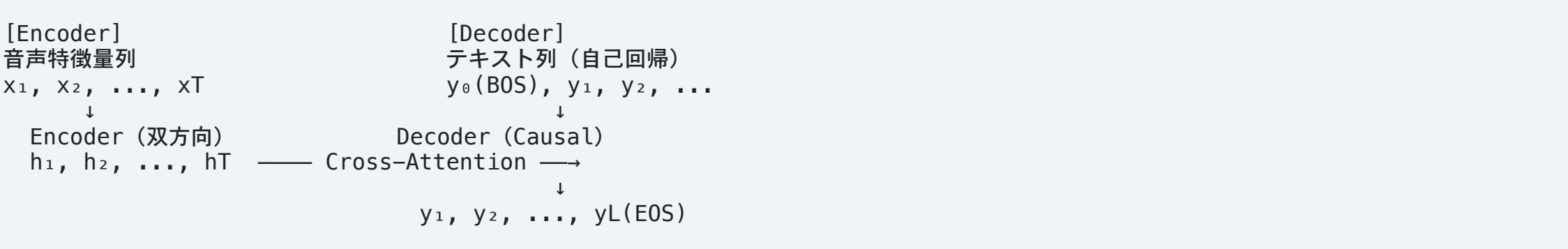
すべての有効なパスの確率の和を最大化する：

$$\mathcal{L}_{\mathrm{CTC}} = -\log P(Y \mid X) = -\log \sum_{\pi \in \mathcal{B}^{-1}(Y)} \prod_{t=1}^{T} p(\pi_t \mid X)$$

特徴まとめ

メリット	デメリット
単純・実装しやすい	条件付き独立性の仮定
ストリーミング認識に向く	内部言語モデルが弱い
前向き・後ろ向きアルゴリズムで効率計算	—

Attention-based Encoder-Decoder（AED）



Attention ウェイト α_{ij}：デコーダが**どのフレームに注目するか**を動的に決定

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T} \exp(e_{ik})}, \quad \mathbf{c}_i = \sum_{j=1}^{T} \alpha_{ij} \mathbf{h}_j$$

メリット：長距離依存関係を捉えやすい、言語的な文脈を自然に組み込める

Transformer と Self-Attention の復習

RNN の課題 → Transformer が解決

課題	Transformer の解決策
長距離依存関係の学習が困難	Self-Attention ですべての位置間を直接計算
逐次処理で並列化できない	並列処理が可能

$$\mathrm{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

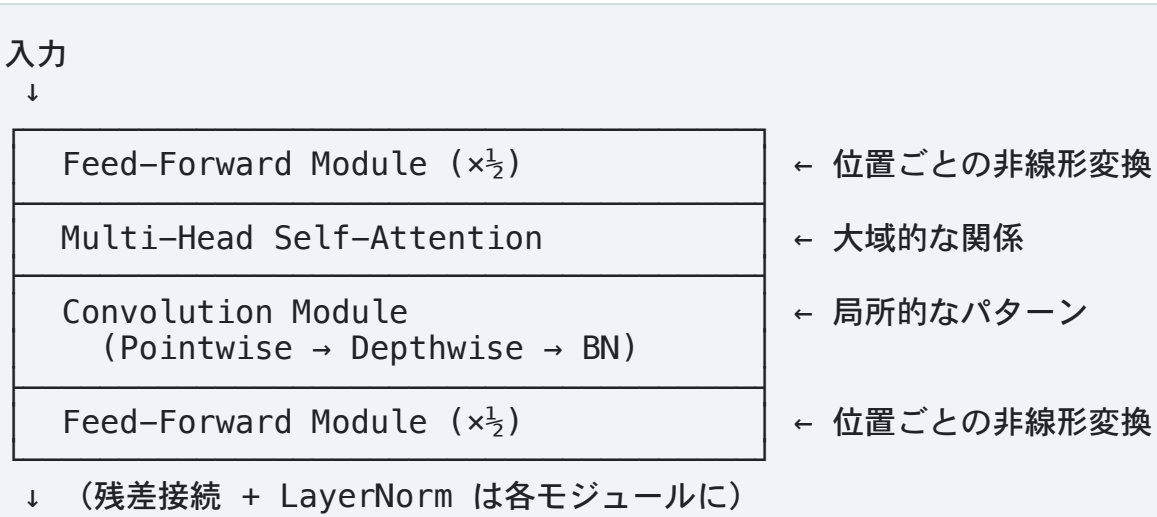
音声での課題：フレーム数が多い（1000フレーム以上）→ $O(T^2)$ の計算コスト

→ Conformer がこの課題を解決する

Conformer — 大域 × 局所を組み合わせる

Transformer の強み：大域的（長距離）な依存関係の捕捉
CNN の強み：局所的な音響パターンの捕捉

→ 両方を組み合わせたのが Conformer！



Conformer の性能

Gulati et al. (2020, Google) — LibriSpeech ベンチマーク

モデル	test-clean WER	test-other WER
Transformer	2.1%	4.9%
Conformer (S)	2.1%	5.0%
Conformer (M)	2.0%	4.6%
Conformer (L)	**1.9%**	**3.9%**

> Conformer は現在も多くの ASR システムのエンコーダとして採用されている。

Whisper とは

Radford et al. (2022, OpenAI) による大規模多言語音声認識モデル

主な特徴

- **680,000 時間**の多言語・多タスク音声データで学習（Web 収集）
- 99 言語の音声認識・翻訳・言語識別を**単一モデル**で実現
- 教師あり学習（ラベル付き大規模データ）

```
680,000時間のデータ内訳（概算）：
┌──────────────────────────────────────────┐
│ 英語音声認識:       438,000 時間          │
│ 多言語音声認識:     125,000 時間（98言語）│
│ 英語への翻訳:       117,000 時間          │
└──────────────────────────────────────────┘
日本語：約 7,000 時間相当を含む
```

Whisper のアーキテクチャ

Encoder-Decoder（Transformer ベース）

```
入力音声（30秒ウィンドウ）
   ↓ ログメルスペクトログラム（80チャネル）
[Convolutional Stem]
  2層Conv + GELU（時間次元を1/2に圧縮）
   ↓
[Transformer エンコーダ]（Nブロック）
   ↓
[Transformer デコーダ] + 特殊トークンによるタスク制御:
    <|startoftranscript|>
    <|ja|>  ← 言語指定
    <|transcribe|> or <|translate|>  ← タスク指定
    <|notimestamps|>  ← オプション
   ↓
[出力テキスト（自己回帰）]
```

Whisper のモデルサイズ

モデル	パラメータ数	メモリ目安	推論速度
tiny	39M	~1GB	32x
base	74M	~1GB	16x
small	244M	~2GB	6x
medium	769M	~5GB	2x
large	1550M	~10GB	1x

Colab 無料 GPU での推奨： `small`（精度とコストのバランス良好）

Whisper の強み・弱み

強み

1. **ゼロショット汎化**：見たことのないドメイン・アクセントでも安定
2. **多言語・多タスク**：認識・翻訳・言語識別を単一モデルで
3. **ロバスト性**：騒音やアクセントへの耐性が高い
4. **タイムスタンプ**：単語レベルの時刻も出力可能（large-v2 以降）

弱み

1. **ハルシネーション**：沈黙・ノイズ区間に存在しないテキストを出力
2. **固定 30 秒ウィンドウ**：長音声では文脈が途切れる
3. **リアルタイム性**：大型モデルはリアルタイムに届かない場合がある
4. **専門用語**：医療・法律などの用語は苦手なことがある

ASR フレームワーク一覧

フレームワーク	開発元	特徴
ESPnet	CMU / 国内大学連合	研究向け。多様な E2E モデルを実装。日本語サポート充実
SpeechBrain	Mila / 大学共同	PyTorch ベース。ASR/TTS/SER 等に対応
Kaldi	JHU 等	従来型 GMM-HMM の標準。業界標準として長く使われた
fairseq	Meta AI	wav2vec 2.0, HuBERT 等の公式実装
HuggingFace	HuggingFace	多数の事前学習モデルを統一 API で利用可能

```
from transformers import pipeline
asr = pipeline("automatic-speech-recognition",
               model="openai/whisper-small", device="cpu")
result = asr("audio.wav")
print(result["text"])
```

実習：Whisper を使ってみよう（Step 1〜3）

Step 1：インストール

```
!pip install openai-whisper
!apt install ffmpeg
```

Step 2：ライブラリのインポートと GPU 確認

```
import whisper
import torch

print("CUDA available:", torch.cuda.is_available())
```

Step 3：モデルのロード

```
model = whisper.load_model("small")
print(f"パラメータ数: {sum(p.numel() for p in model.parameters()):,}")
```

実習：音声ファイルの認識（Step 4）

Step 4：ファイルから認識

```
# 音声ファイルをColabにアップロードしてから実行
result = model.transcribe("audio.wav", language="ja")
print("認識結果:", result["text"])

# タイムスタンプ付きで表示
for segment in result["segments"]:
    start = segment["start"]
    end   = segment["end"]
    text  = segment["text"]
    print(f"[{start:.1f}s → {end:.1f}s] {text}")
```

language="ja" を指定することで日本語認識を確実にする。
省略すると自動言語検出が働く（混合音声では指定を推奨）。

実習：マイク録音して認識（Step 5）

```
from IPython.display import Javascript
from google.colab import output
from base64 import b64decode
import io, soundfile as sf

def record_audio(sec=5):
    print(f"{sec}秒間録音します... 話してください！")
    display(Javascript("""
      var record = time => new Promise(async resolve => {
        stream = await navigator.mediaDevices.getUserMedia({ audio: true })
        recorder = new MediaRecorder(stream)
        chunks = []
        recorder.ondataavailable = e => chunks.push(e.data)
        recorder.start()
        await new Promise(r => setTimeout(r, time))
        recorder.onstop = async () => {
          blob = new Blob(chunks)
          reader = new FileReader()
          reader.onloadend = e => resolve(e.srcElement.result)
          reader.readAsDataURL(blob) }
        recorder.stop() })
    """))
    s = output.eval_js("record(5000)")
    b = b64decode(s.split(",")[1])
    audio_data, sr = sf.read(io.BytesIO(b))
    sf.write("recorded.wav", audio_data, sr)
    return "recorded.wav"

audio_file = record_audio(sec=5)
result = model.transcribe(audio_file, language="ja")
print("認識結果:", result["text"])
```

実習：オプション比較と精度確認（Step 6〜7）

Step 6：オプション比較

```
audio_path = "audio.wav"
result_ja   = model.transcribe(audio_path, language="ja")          # 日本語固定
result_auto = model.transcribe(audio_path)                          # 言語自動検出
result_en   = model.transcribe(audio_path, task="translate")        # 英訳

print(f"日本語認識: {result_ja['text']}")
print(f"自動検出言語: {result_auto.get('language', '不明')}")
print(f"英語翻訳:   {result_en['text']}")
```

Step 7：CER で精度確認

```
!pip install jiwer
from jiwer import cer

reference  = "今日の天気はとても良くて気持ちがいい"    # 実際に話した内容
hypothesis = result_ja["text"].strip()

print(f"正解: {reference}")
print(f"認識: {hypothesis}")
print(f"CER（文字誤り率）: {cer(reference, hypothesis):.1%}")
```

演習：Whisper の誤りパターン分析

以下の5条件で録音・認識し、誤りのパターンを観察する。

条件	正解文	認識結果	CER	主な誤りの種類
標準	…	…	…%	—
速発話	…	…	…%	語境界の崩壊 など
騒音	…	…	…%	音の誤認識 など
方言	…	…	…%	語彙の置換 など
専門用語	…	…	…%	一般語への置換 など

考察のポイント

- 誤りはランダムか、系統的か？
- 日本語特有の誤り（漢字変換ミス、同音異義語）はあるか？
- モデルサイズ（small vs medium）で誤りの種類は変わるか？

自分のペースで取り組んでください。Zoom 参加中の方は発見を口頭で共有も歓迎。

課題の提出について

提出方法：Google Drive（または メール添付）
締め切り：翌週末 23:59

提出内容：

- 誤りパターン分析表（条件別 CER と誤りの種類）
- 考察（100〜200字程度）

任意追加：最も興味深い誤りパターンを1つ選んで、なぜその誤りが起きたかを推測して書く。

本日のまとめ

1. **End-to-End ASR**：音声→テキストを一括学習。アライメント不要。
2. **CTC**：ブランクラベルにより長さ不一致問題を解決。前向き・後ろ向きアルゴリズムで効率計算。
3. **AED**：Attention 機構でエンコーダ出力を参照しながら自己回帰デコード。
4. **Conformer**：Self-Attention（大域）+ Convolution（局所）の組み合わせで高性能エンコーダ。
5. **Whisper**：680,000 時間の大規模多言語データでの教師あり学習。ゼロショット汎化が強み。
6. **実習**：Whisper を Colab で動かし、誤りパターンを分析した。

次回予告（第9回）

音声認識システムの評価指標（WER・CER・SLUE ベンチマーク）

- WER・CER を数式に基づいて手計算する
- 編集距離（Levenshtein 距離）との関係を理解する
- SLUE ベンチマークによる意味理解の評価

第9回は動画回です。
Google Forms テストがあります。事前に今日の実習でやった CER の計算に慣れておきましょう。